

Assignment 04: Lists, tuples, Dictionaries and Sets

CSci 233: Application Program Development w/ Python

Objectives

- Become familiar with the Python basic **Sequence** object, a **list**
- Understand what it means for an item to be **mutable**, and the difference between mutable **lists** and immutable **tuples** in Python
- Practice using list slices to get sub portions of a list.
- Practice iterating over lists and using list methods and functions.
- Learn how to declare and use a **dictionary**, and especially how a key / value pair storage can be useful in an application.
- Become familiar with dictionary objects, adding and deleting items from a dictionary, and using dictionary methods.
- Look at the **set** data type in Python and how it differs from a list.
- Learn some useful applications of sets, and how to create and modify sets in Python.

Description

This assignment consists of 4 to 8 questions that cover topics from chapter 7 (Lists and Tuples) and chapter 9 (Dictionaries and Sets) of our course textbook.

Python provides a very high-level and useful collection of fundamental data types that you can use in your application programs. This unit's materials and textbook readings give an introduction to 4 of the most useful and basic data types, that you will probably make frequent use of if you do a lot of application development using the Python language. In general, most high-level languages, like Ruby, Rust or C++, will provide equivalent data types you can use in the language, so learning the basic concepts of what these are and how they may be useful in building a program can transfer to many other languages and environments.

Basically a **list** and a **tuple** are examples of **sequence** data types. The difference between the two in Python is that a **list** can be modified to add and remove items after it is created (e.g. it is **mutable**), while a tuple cannot be changed once created (it is **immutable**). Sequences allow for efficient sequential access to a set of data. If you mostly just need to iterate through a set of data from start to finish, sequences are the data type you want to use.

However, many times we need to be able to randomly access some data in an efficient way. A sequence forces you to search through all items until you find the one you want. The **dictionary** data type (called by many names in other languages, like a map, a hash or others) allows for efficient random access to a record of data. A **dictionary** is a key-value storage data type, each “value” or “record” is associated with a “key”. A **dictionary** provides efficient and high performance ability to look up a value from the data storage, if you know the key of the data you need to work with.

Finally we will also use Python **sets** in this assignment. Sets may be least used of the data types mentioned in this unit by application programmers, but when you need a collection and need to ensure that all items in the collection are unique, a **set** can end up being very useful and giving you much better performance than using a **list** for the same purpose.

Assignment Prerequisites and Setup

Before performing any assignment in this class, you need to have the following tasks already completed.

1. You need to have `git` tools installed on your system so that you can successfully clone repositories and create commits.
2. You need to have a GitHub account created.
 - You need to have successfully created a ssh key so that you can authenticate with and push commits back to git.
3. You need a working Python 3 distribution installed on the system you will work on these assignments with, that you can run Python scripts and the Python IDLE interface within.

See our class [Getting Started with Python and Git for Class Assignments](#) for links on setting up these needed development environment tools and configuration.

Assignment Questions

Start by cloning the assignment repository that was created for you when you accepted the assignment. There are 5 mostly empty Python script files in the top level of this assignment named `q01.py` through `q05.py`.

You should answer each of the following questions by writing appropriate Python code to solve the stated task. Once you are satisfied with your answer, you should create a commit and push it back to your GitHub repository for grading. In general you should do each question as a separate commit (see [Git Best Practices](#)).

Each commit you push to GitHub will create a release that will be graded. Look at the **Releases** on the right hand side of your **Code** page to see a summary of your grade results. Also the instructor will give feedback for all assignments in your **Feedback** Pull request. See the **Feedback** pull request listed on GitHub on the **Pull requests** tab.

You can and should test your code locally before creating commits and submitting it to GitHub. As mentioned, since you are now writing functions for your assignments, we are now using the `pytest` unit testing module in your autograder and to run tests against your work. So for example, to run the tests for question 01, from the command line you can do:

```
$ python3 -m pytest -v -s test/test_q01.py
===== test session starts =====
platform linux -- Python 3.14.6, pytest-9.1.1, pluggy-1.6.0 -- /opt/base/bin/python3
cachedir: .pytest_cache
rootdir: /workspaces/assg03-solution
collected 8 items

test/test_q01.py::test_freezing PASSED
test/test_q01.py::test_boiling PASSED
test/test_q01.py::test_intersection PASSED
test/test_q01.py::test_room_temperature_1 PASSED
test/test_q01.py::test_room_temperature_2 PASSED
test/test_q01.py::test_body_temperature PASSED
test/test_q01.py::test_zero PASSED
test/test_q01.py::test_one_hundred PASSED

===== 8 passed in 0.01s =====
```

There should be tests for each question in the `test` subdirectory, named `test_q01.py`, `test_q02.py`, etc. for our class assignments. These will be (some of) the same `pytest` tests that are run by the autograder when you submit your work.

For the applications that you write in questions for an assignment, we are also still using the simple input / output diff comparison to test. These are invoked as part of the `pytest` tests of these questions. But as usual, when you are writing the applications, you can look for the expected output in the `test/data` subdirectory, to see exactly what your prompts and output should be formatted to look like in order to pass your tests. For example, if you run the tests for a question 03 application (and you have them passing), you should see:

```

$ python3 -m pytest -v -s test/test_q03.py
===== test session starts =====
platform linux -- Python 3.14.6, pytest-9.1.1, pluggy-1.6.0 -- /opt/base/bin/python3
cachedir: .pytest_cache
rootdir: /workspaces/assg03-solution
collected 5 items

test/test_q03.py::test_small_loan PASSED
test/test_q03.py::test_medium_loan PASSED
test/test_q03.py::test_large_loan PASSED
test/test_q03.py::test_zero_interest PASSED
test/test_q03.py::test_q03_application
Question <q03_application> Test 01: PASSED
Question <q03_application> Test 02: PASSED
Question <q03_application> Test 03: PASSED
Question <q03_application> Test 04: PASSED

=====
All attempted question tests passed (004 passed of 004 tests attempted)
PASSED

===== 5 passed in 0.10s =====

```

Here the q03_application tests are being invoked by the pytest module, but you can still run these by hand if you wish:

```

$ python3 test/test-question.py -c q03_application.py

Question <q03_application> Test 01: PASSED
Question <q03_application> Test 02: PASSED
Question <q03_application> Test 03: PASSED
Question <q03_application> Test 04: PASSED

=====
All attempted question tests passed (004 passed of 004 tests attempted)

```

Question 1: Name Search (15 points)

In this question you will first write some functions that will read a file of records into a list of tuples. Each record of a file will be of the form

```

1
Item
5
Item
3
Item
...

```

Where the first line is an integer which is the rank or popularity of the item, and the second line is the item name, a string. There are data files given to you named `test/data/boy-names.txt` and `test/data/girl-names.txt` which are of this format, but any set of ranked number / item records like this could be used by the application you will develop in this question.

Question 1 Part 1: Create functions to read and process ranked pair lists

Start by writing a function named `load_ranked_items()`. This function will take a filename as its only input parameter. It should return a list of tuples by reading in the file. Each record is expected to have 2 items in it, an

integer on a line, followed by a string item name on the next line. You should process the file line by line in that expected format. You don't have to worry about incorrectly formatted data, but if you are given a filename that does not exist, your function should raise a `FileNotFoundError` exception, which should actually happen without you doing anything if you are opening the file for reading as expected for this function.

The next function will sort your list of tuples so that the highest ranked item is at index 0, the next highest at index 1, etc. **Hint** you should make use of the `sorted()` function here, and not `sort()` the list in place. By default, both `sort()` and `sorted()` will handle a list of tuples as we need here, e.g. default to sorting by the first item in the tuple in ascending order, which should be your rank for each item in the pair. Call your function `sort_ranked_items()`. This function should take the unsorted list of tuples as input. It will return a new list, of the items now sorted by their rank. The tests of this function make sure that you are not modifying the original list, but returning a new sorted list of tuples.

Finally create a method called `get_ranked_items()`. This function will take a sorted list of tuples. It assumes that all ranks from 1 to N (the number of items in the list) are present and are in the sorted list in indexes 0 to $N - 1$. This function takes 2 additional parameters, a `begin` and `end` rank. Use list slicing to get the asked for ranks from the sorted list, and return the new sublist thus asked for. Users do not think like programmers, so if they ask for begin rank 1 through end rank 5, you should be returned the first through 5 ranked items on your list, so adjust `begin` and `end` parameters accordingly to slice your sorted list correctly. This function does need to check that the asked for items exist, and it should raise an `Exception` if they do not. For example, `begin` should be greater than or equal to 1 and should be equal to or less than `end`. Also the neither one can be greater than the largest rank in the list of ranked items.

Question 1 Part 2: Write an application to read and query ranked items pairs

You should write an application in the `q01_application.py` file that allows the user to read in a file of ranked item pair records, sorts the list of tuples, then gives a loop to allow the user to query the ranked items.

You should, as usual, look at the `test/data/q01_application-XX-expected.out` files to get the exact prompts and output to generate. The following are requirements for this application:

- You will need to correctly import your functions for reuse from the `q01.py` file.
- You should validate the input file name. If on trying to load the file, a `FileNotFoundException` is generated, you have to ask the user to input the file name again.
- Likewise, when you query the ranked items list, if it generates an exception because of bad begin or end ranks, you should tell the user to try again.
- The user can query the ranked items until they enter a 0 to indicate they are done with the application.

Question 2: File Analysis with Word Sets (10 points)

In this question, the goal is to write an application to compare the contents of two files to determine various properties about the words in the files. The input files are a simple list of words, each word is on a separate line and all punctuation has been removed and all words are lower case only. Write the following functions to pass the tests given for this question.

Question 2 Part 1: Create functions to create and analyze word sets

Implement the following two function in the file named `q02.py` in your assignment project directory.

Write a function named `load_word_set()`. This function will take the name of a file as input. The file is a simple list of words, one word per line. You should open up and read all of the words. Add each word into a Python `set` and return this set as the result of this function. Because you are creating and using a `set`, the result will be that only the unique words in the file will be recorded in the set that is created and returned. As with previous questions, a `FileNotFoundException` may be generated from this function if you are asked to open a non existent file, which is the expected behavior for your method.

Write a second function named `compare_word_sets()`. This function should take two sets as parameters, lets call them `left_set` and `right_set`. A third parameter that is a string will be passed in, that will be one of the following 'BOTH', 'EITHER', 'LEFT' or 'RIGHT'. Your function will perform the following task based on which value of the third parameter is given:

- ‘BOTH’: create a new set and return it with the words that appear both in the left and the right sets given as inputs.
- ‘EITHER’: create a new set that has all of the words that appear in either the left or right set (or in both sets).
- ‘LEFT’: create a new set that contains the words that appear in the left set, but not in the right set.
- ‘RIGHT’: create a new set and return it of the words in the right set but not in the left set.

If an unknown string is given, your function should throw an exception stating “Unknown comparison given: {comparison}”

Question 2 Part 2: Write an application to analyze file word sets

Write an application that uses your functions for comparing word sets in the file named `q02_application.py`.

Your application should first ask for a left and right file name to load in as sets using your `load_words_set()` function. Use input validation for both file names to ensure that the files exist and can be read in.

Then write a small loop that prompts the user to specify which type of file comparison to perform, and displays the resulting comparison (‘BOTH’, ‘EITHER’, ‘LEFT’, ‘RIGHT’). Your main loop should keep asking the user what to do until they enter ‘quit’. Also instead of performing input validation, simply handle any exception thrown by your compare word sets function, and display an error message.

NOTE: We were expecting you to simply print out the resulting `set()` returned from your comparison as output. However, the order of set items is not guaranteed to be in any particular order. So when you display your set results, use the `sorted()` function again to ensure that the output set results are in sorted ascending order. This should allow you to pass the application tests.

Question 3: E-mail Contact Database (15 points)

In this question, you will be asked to write functions again to implement a small application to maintain a database of names of people and their e-mail addresses. You will use a Python dictionary as the main data structures, where the person’s name is the key, and the value will be their e-mail contact address. Both the key and value used in this application will be string types.

The dictionary will be stored and loaded to and from a file by using file pickling of the dictionary object. See the discussion in chapter 9.3 on serializing objects using pickling. Basically, as you will see by answering this question, pickling allows you to create a binary file relatively easily, much more complex than the simple line based text files we have been using up to this point. A pickled dictionary can act as a simple database for a simple application, providing permanent storage across runs of your application.

Question 3 Part 1: Create functions for dictionary database application

As you should be familiar with at this point, we will start by designing and implementing a small set of functions that will be used to implement the desired e-mail database application. Write the following functions in the `q03.py` file to pass the tests given for them.

Write a function named `store_database()`. This function will take a filename, and a Python dictionary as inputs. Any dictionary could be pickled by this function, though of course for our application it will be a dictionary of names with e-mail addresses as values. The function should always successfully store the function to the indicated filename using pickling. If the file does not exist, it will be created. If the file already exists, it will be overwritten by the new set of data in the dictionary being stored. So no exceptions are expected to be raised when this function is called.

Write another function named `load_database()` that will perform the reverse of storing a dictionary to be used as a database. This function takes a filename as input. It should unpickle the dictionary from the file, and return the loaded dictionary as the result from the function. Your application will expect this function to raise the typical `FileNotFoundError` if the requested file cannot be found to be loaded.

The next 3 functions will be wrappers around basic dictionary functionality to add, modify and remove key/value pairs from a dictionary. We wrap so that we can error check results.

Write a function named `add_contact()`. The functions takes a dictionary, a person’s name as a string, and their e-mail address as a string. Add the new key/value pair contact to the dictionary. But first check that the contact

name does not already exist. If the asked for name is already in the dictionary, a `KeyError` should be raised to indicate an invalid attempt is being made to add in an already existing contact.

The `modify_contact()` function should instead be used if the user wants to change the e-mail address for a contact. This function again takes the dictionary, and a name and e-mail address as strings as input. This function should first check that the contact name already exists in the dictionary database. If the contact does not exist, then again a `KeyError` exception should be raised. Otherwise the indicated contact e-mail address should be updated to the newly given e-mail address.

Implement a `delete_contact()` function. This function takes the dictionary database, and the name of the contact that is to be removed. Again this function should check that the asked for name exists, and if it doesn't it should raise a `KeyError`. Otherwise the asked for contact should be removed from the dictionary database.

Question 3 Part 2: Write the e-mail contact application using your functions

Write the contact database application in the `q03_application.py` in your project directory. Start by prompting the user for the name of the database file to use. Allow the user to specify 'NEW' if they want to create a new contact database. Validate the input here, and do not proceed until either a new database is created, or an existing database pickle file has been loaded for use.

In the main loop, provide the following options for the user:

1. Search the database
2. Add a new contact
3. Modify an existing contact
4. Delete a contact
5. Display all contacts
- Q. Enter Q to quit

Choose action 1-5, Q:

See the 'test/data/q03_application-XX-expected.out' files for the exact prompts and output you need. Each choice should be mostly self explanatory. You are required to reuse your application functions to add, modify and delete contacts from the database dictionary. When the user quits, save the database back to the file used to load the database from. If a 'NEW' database was created, the user should be prompted for a file name to save to before exiting.

Assignment Conclusion / Checklist

Hopefully you have created commits after completing each question and pushed them successfully to your assignment repository. Make sure that you do the following for this and all class assignments.

- Check your autograder results after pushing your commits. Look in your **Feedback** pull request, and in the detailed autograder report generated for your release(s) made for each commit.
- You should never close or merge your **Feedback** pull request, as is mentioned in the comment. The instructor will evaluate your assignments, and may give you feedback about your code or work. Check here after the assignment is returned for a code review and assignment comments.
- In general this class will ask you to follow and use good Python style as specified by the official [Python PEP 8 Style guide](#). You should at least pay attention to
 - All indentation is required to be 4 spaces with no tabs in files.
 - Maximum line lengths should be usually observed, do not generally extend code past the 79 character column in a file.
 - Follow the suggestions for breaking long lines in code (e.g. line break before binary operators in a long expressions).
 - Prefer single quotes for all string in code for class assignments, unless you need a string with a single quote, for example to add an apostrophe 's.
 - Follow Pep8 style for whitespace in expressions and statements.
For example, usually a single space should go before and after all binary operators in expressions.
 - We encourage the use of function annotations in Python code.
- Also in general in this class we will ask you to follow [Git Commit Best Practices](#)

- Strive to use [Meaningful Names](#) for variables, constants and functions in your code. You are required to follow Pep8 naming guidelines, which means **snake_case** names for variables and functions and **SCREAMING_SNAKE_CASE** for constants. Python style switches to **PascalCase** for class names.

Make sure you are checking review comments and code review given for your class assignments. We may start by making suggestions where your style or practices could be improved. As the class progresses, some of these suggestions may become requirements, especially for students who are repeatedly making the same style or practice error and are not following feedback to correct a noted issue in future assignments. Assignments may be left ungraded in some cases for style or practice issues until corrected, and/or may have points removed or receive a 0 grade if issues continue after receiving multiple feedback that are repeatedly ignored and not corrected.

Additional Information

The following are suggested online materials you may use to help you understand the tools and topics we have introduced in this assignment.

- [Python PEP 8 Style guide](#)
- [Git Commit Best Practices](#)
- [Git Best Practices](#)
- [Best Practices to Write Readable and Maintainable Code: Choosing meaningful names](#)
- [How to Write Better Names for your Variables, Functions and Classes](#)